Linux e terminali 2

Mattia Graziani, Alice Benatti

20 aprile 2026

>ADM staff

Laboratori fra pari

Standard Input/Output e ridirezioni

Abbiamo iniziato a vedere l'esistenza di operazioni un po' più potenti dei singoli comandi. Infatti combinando due o più comandi possiamo creare complesse procedure.

Per combinare comandi è importante ricordarsi che:

- Un comando restituisce sempre (o quasi, es `cd`) qualcosa sullo *standard output*.
- Lo *standard output* è considerato come un file (in memoria) dal sistema.
- La maggior parte dei comandi visti fino ad adesso hanno la possibilità di prendere in input lo *standard output* invece che un file classico.

Combinare comandi

Come abbiamo visto è possibile mandare l'output di un comando nell'input di un altro comando grazie alla **pipe** |

Esiste anche l'**operatore di ridirezione** > che al posto di indirizzare l'output in un comando, scrive (sovrascrive) direttamente su un file.

Sintassi: `comando > file`

Combinare comandi

Possiamo usare il comando di redirezione > anche più volte in modo concatenato.

```
[mattia@pirandello prove]$ ls
capybara  castoro  criceto  file
[mattia@pirandello prove]$ ls > file1 > file2
[mattia@pirandello prove]$ ls
capybara  castoro  criceto  file  file1  file2
[mattia@pirandello prove]$ cat file1
capybara  castoro  criceto  file  file1  file2
[mattia@pirandello prove]$ cat file2
capybara  castoro  criceto  file  file1  file2
[mattia@pirandello prove]$
```

Ridirezione su file non distruttiva

Esiste anche un operatore per indirizzare su file l’output di un comando senza sovrascrivere il contenuto del file, ma "appendendo" alla fine del file il contenuto scritto.

Sintassi: `comando » file`

Ridirezione dello standard error

Di predefinito l'operatore di redirezione reindirizza solo lo *standard output*. Questo significa che se un comando restituisce un errore, questo non verrà redirezionato:

```
[mattia@pirandello prove]$ ls /asdkjc > mangusta
ls: cannot access '/asdkjc': No such file or
directory
[mattia@pirandello prove]$
```

Figura: Esempio di *stderr* non ridirezionato

Per ridirezionare anche lo *standard error* dentro il file è necessario specificarlo alla shell.

Ridirezione dello standard error

Lo *standard error* in linux ha come *file descriptor* il **2**. Non importa sapere cosa sia un file descriptor, basta ricordarsi che **stdout = 1** e **stderr = 2**.

Per ridirezionare lo *stderr* sullo *stdout* si usa la seguente sintassi: **2>&1**

Redirezionando lo *standard error* verso lo *standard output* è possibile scrivere entrambi su file:

```
[mattia@priandello prove]$ ls /asdkjc > mangusta 2>&1
[mattia@priandello prove]$ cat mangusta
ls: cannot access '/asdkjc': No such file or
directory
```

Figura: Esempio di *stderr* ridirezionato

Ridirezione dello standard error

Da notare che questo funziona anche per la pipe:

```
[mattia@priandello prove]$ ls /asdkjc | head -c 10
ls: cannot access '/asdkjc': No such file or
directory
[mattia@priandello prove]$ ls /asdkjc 2>&1 | head -c 10
ls: cannot
[mattia@priandello prove]$
```

Figura: Esempio di *stderr* ridirezionato

/dev/null

Come si fa a ridirezionare soltanto lo *stderr* su file e "buttare via" lo *stdout*?

Esiste in Linux un "file" che indipendentemente da cosa gli passate lo butta via. Questo "file" è **`/dev/null`**.

```
[mattia@pirandello prove]$ ls /asdkjc / 2> /tmp/pippo
1> /dev/null
[mattia@pirandello prove]$ cat /tmp/pippo
ls: cannot access '/asdkjc': No such file or
directory
```

Figura: Esempio di stderr ridirezionato

Ricordiamoci che `stdout = 1` e `stderr = 2`.

Filtri

Quando si combinano assieme più comandi spesso i seguenti tornano utili:

- `sort`: Restituisce in *stdout* il testo ordinato preso in *stdin*
- `uniq`: Se ci sono più linee consecutive uguali ne lascia solo una
- `grep`: Cerca il testo passato come argomento
- `wc`: Conta linee, parole e caratteri passati in input
- `head`: Manda in *stdout* solo la prima parte del testo
- `tail`: Manda in *stdout* solo l'ultima parte del testo

echo, echo, echo...

Il comando `echo`, come suggerisce il suo nome, serve per stampare quello che gli viene passato come argomento:

```
[mattia@pirandello prove]$ echo "pippo"
pippo
[mattia@pirandello prove]$ echo "Questa è una frase"
Questa è una frase
[mattia@pirandello prove]$
```

Figura: Esempio di echo

A prima vista può sembra abbastanza inutile, in realtà ha tantissime applicazioni.

Seeing the world as the shell sees it

```
[mattia@pirandello prove]$ echo *
1 capybara castoro criceto file file1 file2 mangusta
[mattia@pirandello prove]$ echo */lontra
capybara/lontra castoro/lontra
```

Figura: Esempio di echo con wildcard

Come possiamo vedere `echo` non stampa * ma lo tratta come la shell: ovvero come una *wildcard*.

In realtà è proprio la shell a sostituire la wildcard con le entries opportune prima di eseguire il comando `echo`.

Seeing the world as the shell sees it

Esiste una particolare sintassi per indicare un range in bash:

```
[mattia@pirandello prove]$ echo {1..11}
1 2 3 4 5 6 7 8 9 10 11
[mattia@pirandello prove]$ echo {c..t}
c d e f g h i j k l m n o p q r s t
```

Figura: Esempio di echo range

La shell, prima di eseguire l'echo, sostituisce le parentesi graffe con il range specificato.

Non funziona solo con echo in quanto è la shell stessa a fare la sostituzione. Funziona molto bene con `mkdir`:

```
mkdir {2020..2021}-{01..12}-{01..31}
```

Seeing the world as the shell sees it

E si usa come i metalinguaggi dei prof:

```
[mattia@pirandello prove]$ echo {1,11}
1 11
```

Figura: Esempio di due elementi di un range

```
[mattia@pirandello prove]$ echo A{dm,pp}Staff
AdmStaff AppStaff
```

Figura: Esempio con stringhe

La pipe non riesce a fare tutto

Con la pipe **|** possiamo combinare più comandi assieme, però non riusciamo a fare ancora tutto. *Se volessimo vedere il tipo di tutti i file che hanno la parola zip nel nome dentro la cartella `/bin` come facciamo?*

```
[mattia@pirandello prove]$ ls -d /bin/* | grep zip | file
Usage: file [...]
```

Figura: Esempio **SBAGLIATO**

Questo comando restituisce un errore perché il comando `file` non riesce ad interpretare lo standard input tramite la pipe, infatti `file` vuole gli input come argomenti: `file ...`

Sostituzione di comandi

La shell ammette una sintassi per la sostituzioni di comandi all'interno di un'espressione.

Sintassi: `$(comando)`

La shell sostituirà l'intera espressione al risultato del comando:

```
[mattia@pirandello prove]$ echo $(ls)
1  capybara  castoro  criceto  file  file1  file2  mangusta
```

Figura: Esempio di sostituzione del comando `ls`

Sostituzione di comandi

Ritornando a quello che volevamo fare prima: *Se volessimo vedere il tipo di tutti i file che hanno la parola zip nel nome dentro la cartella `/bin` come facciamo?* **Con la sostituzione di comandi!**

```
[mattia@pirandello prove]$ file $(ls -d /bin/* | grep zip)
```

Figura: Esempio di più comandi

Matematica base

Si possono fare anche operazioni matematiche con questo tipo di sostituzione:

```
[mattia@pirandello prove]$ echo $((1 + 1))
2
[mattia@pirandello prove]$ echo $((9 * 5))
45
[mattia@pirandello prove]$ echo 3GB are $((3 * 1024))MB
3GB are 3072MB
```

Figura: Esempio di più comandi

Nota: è necessario usare la doppia parentesi `(())` in quanto vogliamo che sia un'espressione aritmetica.

Variabili

È possibile salvare dei valori in delle **variabili**.

Alcuni programmi sono configurabili attraverso le **variabili di ambiente**.

Cerchiamo di capire la differenza tra variabili e variabili d'ambiente

```
[mattia@pirandello prove]$ echo $USER
mattia
[mattia@pirandello prove]$ echo $SHELL
/bin/bash
```

Figura: esempi variabili già esistenti nella shell

Assegnamento delle variabili

Per assegnare un valore ad una **variabile** si può usare la seguente sintassi:

`varname=value`.

```
[mattia@pirandello prove]$ pluto=cane
[mattia@pirandello prove]$ echo $pluto
cane
```

Figura: Esempio di assegnamento variabile

Variabili

Una variabile è vuota se, quando si "printa" con `echo`, non ha nulla al suo interno.

```
[mattia@pirandello prove]$ echo $pippo

[mattia@pirandello prove]$ echo $aajsdklsakjd

[mattia@pirandello prove]$
```

Uso di variabili

Si possono utilizzare le variabili nei comandi:

```
[mattia@pirandello prove]$ src=castoro
[mattia@pirandello prove]$ dest=capybara
[mattia@pirandello prove]$ cp -r $src $dest
```

Figura: Esempio di copia utilizzando variabili

```
[mattia@pirandello prove]$ programs=$(ls /bin)
[mattia@pirandello prove]$ echo $programs | less
```

Figura: Tutti i programmi in `/bin` sono salvati nella variabile *programs*

La variabile PATH

Una variabile spesso usata è la variabile di `PATH`.

Quando digitiamo un comando la shell andrà a cercare se esiste un eseguibile con lo stesso nome del comando nelle directory indicate dalla variabile `PATH`.

```
[mattia@pirandello prove]$ echo $PATH
/usr/local/bin:/usr/local/sbin:/usr/bin:/usr/l
ib/jvm/default/bin:/usr/bin/site_perl
[mattia@pirandello prove]$
```

Figura: Percorsi nella variabile `PATH`

Variabili vs Variabili d’ambiente

Abbiamo visto degli esempi di semplici variabili, come tutte quelle che abbiamo creato, e di variabili d’ambiente, tipo `$PATH`.
Alla fine non c’è molta differenza, di predefinito le variabili non sono propagate anche sui processi figli della shell. Le variabili d’ambiente invece sono proprio quelle che sono viste globalmente.

Se aprite una nuova shell e stampate una variabile che avete creato prima la vedrete vuota, invece `$PATH`, come le altre variabili d’ambiente, sarà sempre visibile.

Export

È possibile, e spesso necessario, esportare le variabili per renderle visibili dai processi figli della shell. (processo = attività di un programma svolta sul processore)

```
[mattia@pirandello prove]$ pippo=cane
[mattia@pirandello prove]$ echo $pippo
cane
[mattia@pirandello prove]$ bash -c 'echo $pippo'

[mattia@pirandello prove]$ export pippo=cane
[mattia@pirandello prove]$ bash -c 'echo $pippo'
cane
```

Figura: Percorsi nella variabile `PATH`

htop

Linux è sistema che permette l'esecuzione di più processi simultaneamente.

Per vedere i processi in esecuzione (e altre informazioni) è molto comodo il comando `top` o `htop` (la versione nuova).

Se si vuole invece aver uno snapshot (istantanea) dei processi in esecuzione si può utilizzare il comando `ps`.

Per vedere tutti i processi su `ps` la flag è `-e`.

Pid e kill

Per identificare in modo univoco un processo si utilizza un intero positivo chiamato **pid**: Process IDentifier. Il pid può essere ottenuto attraverso il comando `ps`.

Se per caso volessimo terminare un processo lo si può fare attraverso il comando `kill` passando il pid del processo da uccidere.

```
[mattia@pirandello prove]$ ps
    PID TTY          TIME CMD
   8239 pts/3    00:00:03 zsh
  20813 pts/3    00:00:00 ps
[mattia@pirandello prove]$ kill 8239
```

Figura: Esempio di `kill`

Quante shell!

Esistono varie shell:

- `sh` (Bourne Shell)
- `bash` (Bourne Again Shell)
- `zsh` (Z sh, da Zhong Shao)
- `fish` (Friendly Interactive sh)
- `csh`
- `tcsh`
- `dsh`
- `ksh` ...

Tutte servono per interagire con il sistema operativo, però sono tutte leggermente diverse sotto alcuni aspetti.

Di solito la predefinita nei sistemi Linux è `bash`.

Bash scripting

Spesso si vogliono automatizzare dei comandi della shell, oppure eseguirne più in "una volta sola".

La shell (e in particolare bash) permette di scrivere degli script.

Uno **script** è un file di testo con un elenco di comandi che vengono eseguiti in sequenza da bash (dall'interprete che specificate) .

Per specificare che interprete da usare è necessario indicare nella prima riga del file il percorso assoluto all'eseguibile della shell:

```
#!/bin/bash
```

Figura: Riga per indicare bash come interprete

Creare un nuovo script

Per eseguire uno script bash serve un file (lo script). Per convenzione gli script bash si salvano con estensione **.sh**

```
[mattia@pirandello prove]$ nano script.sh
```

```bash
#!/bin/bash
echo "Reading programs"
PROG=$(ls -d /bin/*)
NUM=$(echo "$PROG" | grep "ca" | wc -l)
echo "Numero di programmi con 'ca' è $NUM"
```

Figura: Cosa fa?

Esempio di script

Questo script fa le seguenti operazioni:

1. Stampa sullo *standard output* "*Reading programs*"
2. Salva tutti i programmi presenti in `/bin` nella variabile `PROG`.
3. Salva nella variabile `NUM` il numero di tutte le stringhe presenti nella variabile `PROG` che hanno al loro interno "ca".
4. Stampa il valore presente nella variabile `NUM`.

Eseguire uno script

Come abbiamo visto la scorsa volta, bisogna rendere lo script eseguibile aggiungendogli il permesso: `chmod +x script.sh`

```
[mattia@pirandello prove]$ ls -al
-rw-r--r-- 1 mattia mattia Nov 06 18:00 script.sh
[mattia@pirandello prove]$ chmod +x script.sh
-rwxr-xr-x 1 mattia mattia Nov 06 18:00 script.sh
```

Per eseguirlo basta aggiungere `./` prima del nome:

```
[mattia@pirandello prove]$ ./script.sh
```

Figura: Esecuzione di uno script bash

find - ricerca binaria sul filesystem

IDEA: Non cercare contenuti, cerca i contenitori.

```
find [PERCORSO] [FILTRI] [AZIONE]
```

- PERCORSO: L'origine della ricerca (. per la cartella corrente, / per la root).
- FILTRI: I criteri di selezione dei file.
- AZIONE: Cosa eseguire sui risultati (se non specificato, li stampa a schermo).

find - filtri di ricerca

FILTRI: I criteri di selezione dei file.

- Per Tempo (Usa i segni: - meno di, + più di)
  - mtime (Giorni dall'ultima modifica del contenuto).
  - mmin (Minuti dall'ultima modifica).
  - atime / ctime (Accesso o modifica metadati).
- Per Dimensione
  - -size (Es. +1G per "più di 1 GB", -50k per "meno di 50 KB").
  - -empty (Trova file o cartelle completamente vuoti).
- Per Utente / Permessi
  - -user (Cerca per proprietario del file).
  - -perm (Cerca permessi esatti come 777 o parziali come /u=w).

find - operatori logici

Esistono anche operatori logici per combinare più filtri e creare ricerche più complesse.

- AND (Implicito): Basta scriverli di seguito (es. file .log E più grandi di 10MB).
- OR (-o): Ne basta uno vero (es. file .pdf O .doc).
- NOT (!): Inverte il match (es. tutto tranne i file .txt).

find - il vero potere in azione

È possibile manipolare i risultati di find usando l'opzione di esecuzione dei comandi sui file trovati (`-exec`)

- `{}`: Il segnaposto che rappresenta il file trovato.
- `;`: Chiude il comando (lo esegue una volta per ogni file).
- `+`: Chiude il comando accodando tutti i file (lo esegue una volta sola in blocco, molto più efficiente).
- `-delete`: Eliminazione diretta dei file trovati (da usare con estrema cautela)

Qualche esempio:

- Pulizia automatica file vecchi `find /tmp -type f -mtime +30 -delete`
- Rende eseguibili tutti gli script in massa
  `find . -name "*.sh" -exec chmod +x +`

find - ottimizzazione e sicurezza

È importante usare find con attenzione, soprattutto quando si eseguono comandi sui risultati, per evitare di causare danni involontari (es. eliminare file importanti). Alcuni consigli:

- Utilizzo della Gestione Spazi (`-print0`): Stampa i risultati separandoli con un carattere nullo, fondamentale quando si passano i dati a `xargs -0` per non "rompere" i nomi di file con spazi.
- Limiti di Ricerca (`-maxdepth`): Es. `-maxdepth 2` impedisce a find di scendere in un loop infinito di sottocartelle, fermandosi al secondo livello.

ack - ricerca intelligente nel codice

`ack` cerca testo nei file di codice in modo rapido e mirato.

- È pensato per lo sviluppo: di default evita file binari e cartelle non utili.
- Supporta espressioni regolari (PCRE), quindi ricerche più potenti di `grep` base.
- Riconosce i tipi di file e permette filtri pratici per linguaggio.

Sintassi base: `ack "pattern" [percorso]`

Esempi:

- `ack "TODO" .`
- `ack -python "def test" .`
- `ack -i "errore|warning" logs/`

ack - filtri per tipo

`ack` riconosce oltre 50 tipi di file comuni (es. perl, python, html, css) e permette di filtrare facilmente la ricerca per tipo.

- Inclusione specifica: Cerca solo in un determinato linguaggio (es. `-python`).
- Esclusione specifica: Cerca ovunque tranne che in un determinato linguaggio (es. `-nocpp`).
- Esplorazione tipi: Visualizza tutti i formati riconosciuti (`-type-list`).

ack - ricerca avanzata

Oltre alla semplice ricerca di stringhe, `ack` supporta funzionalità avanzate per ricerche più intelligenti e contestuali sfruttando la potenza delle **espressioni regolari** (o *regex*).

- Ricerca contestuale (Lookahead/Lookbehind): Trova parole solo se precedute/seguite da altre (es. `ack "User(?=Profile)" .`).
- Parola esatta (`-w`): Evita i falsi positivi ignorando le sottostringhe (es. trova "root" ma non "bootstrap").
- Case-insensitive (`-i`): Ignora la differenza tra maiuscole e minuscole.

ack - controllo dell'output

`ack` offre diverse opzioni per **gestire e formattare i risultati** della ricerca, rendendo più facile l'analisi dei dati trovati.

- Pulizia visiva (`-l`): Restituisce solo i nomi dei file che contengono il match.
- Statistiche (`-cl`): Conta le occorrenze del match all'interno di ogni file.
- Contesto (`-B` e `-A`): Mostra un numero definito di righe prima (Before) e dopo (After) la stringa trovata.

ack - integrazione e automazione

`ack` è progettato per essere integrato in flussi di lavoro più ampi, permettendo di combinare la potenza della ricerca intelligente con altri strumenti e processi di sviluppo.

- Pipeline con altri comandi: Combinazione ideale con `xargs`, `sed` o `perl` per eseguire operazioni di "Trova e Sostituisci" in massa su interi progetti.
- File di configurazione (`.ackrc`): Salva preferenze globali (es. `-sort-files`, `-smart-case`) e permette la creazione di gruppi di file personalizzati (es. `-type-add=web:ext:php,html,css`).

ack - alternative e successori

`ack` è stato uno dei primi strumenti a portare una ricerca intelligente e specifica per sviluppatori, ma negli ultimi anni sono emerse alternative più veloci e ottimizzate per codebase enormi.

- `ag` (The Silver Searcher): Scritto in C, è più veloce di `ack` su grandi codebase, ma con meno funzionalità di filtro.
- `ripgrep` (`rg`): Scritto in Rust, è attualmente uno degli strumenti più veloci per la ricerca nei file di codice, con supporto nativo per Unicode e un sistema di filtro simile a `ack`.

sed - stream editor

`sed` è un potente strumento di manipolazione del testo che permette di eseguire modifiche complesse su file o flussi di testo in modo non interattivo.

È particolarmente utile per automatizzare la modifica di grandi quantità di testo, come file di configurazione, log o dati strutturati.
La sua filosofia è quella di lavorare riga per riga in modo non interattivo, rendendolo ideale per l'automazione di compiti di editing testuale.
Sintassi di base: `sed [opzioni] <comando> file`
Salvataggio (`-i`):

- `-i`: Applica le modifiche direttamente sul file originario (in-place).
- `-i.bak`: Crea automaticamente una copia di backup prima di sovrascrivere.

sed - selezione delle righe

`sed` permette di selezionare specifiche righe su cui applicare i comandi, rendendo possibile eseguire modifiche mirate senza dover processare l'intero file.

- Per Numero: Es. `3,5` (Applica il comando solo dalla riga 3 alla 5).
- Per Pattern (Regex): Es. `/^#/` (Applica il comando solo alle righe che iniziano con #).
- Per Fine File ($): Es. `10,$` (Dalla riga 10 fino alla fine del documento).

sed - sostituzione e regex

L'uso più comune di `sed` è la sostituzione, che permette di modificare il testo in modo flessibile usando espressioni regolari.

Sintassi base: `s/vecchio/nuovo/` (Sostituisce il testo).
Opzioni avanzate:

- Espressioni Regolari Estese (`-E`): Evita l'uso eccessivo di backslash per caratteri speciali.
- Gruppi di Cattura: Isolano parti di testo usando le parentesi tonde (...). Vengono richiamati nel nuovo testo con i backreference `\1`, `\2`, `\3` ecc. (es. per invertire l'ordine di una data).

sed - comandi operativi

Oltre alla sostituzione, `sed` offre una serie di comandi operativi che permettono di eseguire modifiche più complesse.

- `d` (Delete): Elimina le righe selezionate (es. rimuovere le righe vuote).
- `p` (Print): Stampa le righe selezionate (si usa in combinazione con l'opzione `-n` per silenziare l'output automatico di tutto il resto).
- `a` (Append): Aggiunge una nuova riga di testo *dopo* l'occorrenza trovata.
- `i` (Insert): Inserisce una nuova riga di testo *prima* dell'occorrenza.

sed - gestione della memoria

- Pattern Space: L’area di lavoro principale dove viene caricata la riga attualmente in elaborazione. Tutti i comandi di modifica agiscono su questa area.
- Hold Space: Un buffer secondario che funge da "appunti". Permette di salvare temporaneamente il contenuto del pattern space o di accumulare informazioni per operazioni più complesse.
- Comandi di Interscambio:
  - `h` (hold): Copia il contenuto del pattern space nel hold space, sovrascrivendo qualsiasi cosa ci fosse prima.
  - `g` (get): Sostituisce il contenuto del pattern space con quello del hold space, sovrascrivendo la riga attualmente in elaborazione.
  - `G` (append get): Aggiunge il contenuto del hold space alla fine del pattern space, mantenendo la riga attualmente in elaborazione.

sed - trucchi avanzati e ottimizzazione

`sed` è uno strumento estremamente potente, e con un po' di creatività è possibile sfruttarlo per eseguire operazioni complesse in modo efficiente.

- Delimitatori Custom: Si può sostituire lo slash / con altri caratteri (es. |) per rendere leggibili le modifiche ai percorsi o agli URL (es. `s|/usr/bin|/usr/local|`).
- Concatenazione (`-e`): Permette di eseguire sequenze di comandi multipli in una sola passata (es. cancella i commenti *E* cancella le righe vuote contemporaneamente).

history - gestione avanzata I

La cronologia dei comandi è una potente funzionalità che permette di richiamare, modificare e gestire i comandi eseguiti in passato. Ecco alcune tecniche avanzate per sfruttarla al meglio:

- Richiamo rapido (Event Designators):
  - `!!` (Riesegue l'ultimo comando)
  - `!n` / `!-n` (Riesegue per numero esatto o posizione all'indietro)
  - `!stringa` / `!?stringa?` (Riesegue l'ultimo comando che inizia con o contiene la stringa).
- Recupero argomenti (Word Designators):
  - `!$` (Recupera solo l'ultimo argomento del comando precedente)
  - `!:^` (Recupera solo il primo argomento)
  - `!:*` (Recupera tutti gli argomenti escludendo il comando base).

history - gestione avanzata II

- Ricerca interattiva (Scorciatoie da terminale):
  - Ctrl + R (Avvia la "Reverse Search" per cercare digitando parte del comando all'indietro)
  - Ctrl + G (Interrompe la ricerca senza eseguire nulla).
- Configurazione ambiente (Variabili in `/.bashrc`):
  - HISTSIZE e HISTFILESIZE (Aumentano il limite dei comandi salvati in memoria e su disco)
  - HISTCONTROL (Evita il salvataggio di duplicati o comandi preceduti da spazio)
  - HISTTIMEFORMAT (Aggiunge data e ora ai log)
  - PROMPT_COMMAND (Con `history -a` salva i comandi in tempo reale senza attendere la fine della sessione).

history - gestione avanzata III

- Manutenzione manuale (Opzioni dirette del comando):
  - `-c` (Svuota la cronologia della sessione attuale)
  - `-w` (Forza la scrittura della cronologia sul file `.bash_history`)
  - `-d <numero>` (Elimina chirurgicamente una singola riga dalla cronologia).
- Trucchi avanzati (Privacy e Correzione):
  - Trucco dello spazio (Digitare uno spazio prima del comando per non farlo registrare nei log, se abilitato da HISTCONTROL)
  - Sostituzione Rapida `^vecchio^nuovo` (Corregge al volo un errore di battitura nell'ultimo comando eseguito e lo lancia immediatamente).

tr - chirurgia dei singoli caratteri

Funzionamento Base e Sintassi:

- **Solo Standard Input** - Non accetta file come argomenti, richiede sempre pipe | o ridirezione < (es. `echo "Testo di esempio" | tr 'a-z' 'A-Z'` per convertire in maiuscolo).
- **Mappatura 1:1** - Associa e sostituisce i caratteri dal primo al secondo set (es. `tr 'abc' '123'` trasforma "abc" in "123").
- Sintassi: `tr [opzioni] SET1 [SET2]`

tr - set di caratteri

Set di Caratteri (Classi POSIX):

- `[:upper:]` / `[:lower:]` (Gestione maiuscole e minuscole)
- `[:digit:]` / `[:alnum:]` (Identificazione di numeri e valori alfanumerici)
- `[:space:]` (Gestione di spazi, tabulazioni e ritorni a capo)

tr - opzioni chiave

Opzioni Chiave (I modificatori di comportamento):

- `-d` **Delete** - Elimina istantaneamente i caratteri specificati nel SET1
- `-s` **Squeeze** - Comprime le sequenze di caratteri identici ripetuti in una singola occorrenza, ideale per ripulire spazi extra
- `-c` **Complement** - Inverte la selezione logica: applica l'operazione a tutto ciò che **NON** è presente nel SET1
- `-t` **Truncate** - Tronca la lunghezza del SET1 per adattarla a quella del SET2

tr - casi d'uso frequenti

Casi d'Uso Frequenti (SysAdmin):

- Pulizia Formattazione - Rimuovere ritorni a capo `\n` per appiattire un file su una riga con `-d`)
- Estrazione di Precisione - Isolare solo i numeri in un testo disordinato combinando `-cd` (es. `echo "abc123def456" | tr -cd '[:digit:]'` restituisce "123456").
- Sicurezza - Filtrare `/dev/urandom` tenendo solo caratteri leggibili per generare password)
- Crittografia Base - Traslazione di lettere per creare cifrari come il ROT13

Affrontiamo Vim

La scorsa volta abbiamo visto che su Linux esistono due principali editor di testo: nano e vim.

Cerchiamo di capire perché proprio vim, qual è la sua **filosofia**?

- Quando si programma la maggior parte del tempo è passato a *modificare* il codice, non a scriverlo.
- Modificare il codice include molto altro oltre a scrivere: eliminare, sostituire, riordinare, duplicare, formattare, ecc.
- Ha senso facilitare tutta la parte di modifica del codice, più che di scrittura effettiva.
- Il mouse è una perdita di tempo quando si deve scrivere, se si può fare tutto da tastiera in modo efficiente è meglio.
- Un'operazione usa il minor numero di tasti possibili.

Modalità

In un editor classico ci si trova sempre nella modalità di *modifica*, ovvero se si schiacciano delle lettere vengono inserite nel file che si sta modificando.

Per la maggior parte degli editor (compreso nano) questa è l'unica modalità presente.

Vim introduce tre modalità principali:

- Normal mode
- Insert mode
- Visual mode

Modalità

Vim introduce tre modalità principali:

- **Normal mode**: Modalità predefinita di vim. La si attiva con il tasto `ESC`. Permette di muoversi all'interno del documento in modo rapido. Di eliminare, copiare, incollare e spostare parti di testo.

Modalità

Vim introduce tre modalità principali:

- **Normal mode**: Modalità predefinita di vim. La si attiva con il tasto `ESC`. Permette di muoversi all'interno del documento in modo rapido. Di eliminare, copiare, incollare e spostare parti di testo.
- **Insert mode**: Modalità classica per l'inserimento e la scrittura di testo. È la stessa che si trova nella maggior parte degli editor. Ci si accede principalmente usando il tasto '`i`', ma anche '`I`', '`a`', '`A`', ecc.

Modalità

Vim introduce tre modalità principali:

- **Normal mode**: Modalità predefinita di vim. La si attiva con il tasto `ESC`. Permette di muoversi all'interno del documento in modo rapido. Di eliminare, copiare, incollare e spostare parti di testo.
- **Insert mode**: Modalità classica per l'inserimento e la scrittura di testo. È la stessa che si trova nella maggior parte degli editor. Ci si accede principalmente usando il tasto '`i`', ma anche '`I`', '`a`', '`A`', ecc.
- **Visual mode**: Permette di selezionare parti di testo, sia verticalmente che orizzontalmente. Ci si accede usando il tasto '`v`'.

Normal mode

La **normal mode** è la predefinita e quella su cui si dovrebbe sempre stare se non si necessitano di altre modalità. Per entrarci usare il tasto `ESC`.

Per spostarsi all'interno del testo in normal mode si usano le seguenti lettere e NON le freccette:

- `j`: Per spostarsi verso il basso di una riga
- `k`: Per spostarsi verso l'alto di una riga
- `h`: Per spostarsi a sinistra di un carattere
- `l`: Per spostarsi a destra di un carattere

Il motivo per cui si utilizzano queste lettere e non le freccette è il fatto che lo *spostamento della mano* per raggiungere le freccette è troppo grande e richiederebbe troppo tempo per la filosofia di vim.

Normal mode

Altre combinazioni di tasti base in normal mode:

- `w`: si sposta all'inizio della parola successiva
- `e`: si sposta alla fine della parola successiva
- `b`: si sposta all'inizio della parola precedente
- `gg`: si sposta all'inizio del file
- `G`: si sposta alla fine del file
- `dd`: cancella la riga su cui si trova il cursore
- `yy`: copia la riga su cui si trova il cursore
- `p`: incolla il testo copiato nella riga successiva al cursore
- `x`: elimina il carattere sotto il cursore
- `u`: annulla l'ultima operazione fatta
- `ctrl + r`: ripete l'ultima operazione annullata (il contrario di '`u`')

Insert mode

Si utilizza la **insert mode** quando si vuole scrivere del testo. Per entrarci dalla normal mode si possono usare i seguenti tasti:

- `i`: si inizia a digitare prima della lettera sotto il cursore
- `a`: si inizia a digitare dopo della lettera sotto il cursore
- `I`: si inizia a digitare all'inizio della riga
- `A`: si inizia a digitare alla fine della riga
- `o`: si inizia a digitare in una nuova linea sottostante
- `O`: si inizia a digitare in una nuova linea sovrastante
- `cc`: si cancella la riga su cui si trova il cursore e si inizia a scrivere al suo posto

How to exit vim

Uscire da vim è una delle operazioni più complesse.

Molti dicono che l'unico modo sia riavviare il pc.

In realtà basta sapere come funziona!

Per uscire da vim bisogna trovarsi in **normal mode** e digitare `:q` e successivamente premere `invio`.

Se il file è stato modificato dall'apertura il comando non funzionerà perché vim non sa se vuoi salvare il file o no. Per salvarlo (ed uscire) si usa `:wq` e per non salvarlo (ed uscire) `:q!`

Per salvare quindi un file senza uscire basta `:w`

Creiamo un nuovo script

Apriamo un nuovo file .sh con vim:

```
[mattia@pirandello prove]$ vim newScript.sh
```

Iniziamo a scrivere entrando dalla Normal Mode alla Insert Mode con "i"
In prima riga mettiamo come sempre:

```
#!/bin/bash
```

Variabili d'ambiente e comandi

Proviamo intanto a vedere se i comandi e le variabili d'ambiente che abbiamo usato funzionano:

```
#!/bin/bash

TIMESTART=$(date)

echo "Script started by ${USER} at ${TIMESTART}"
```

Salviamo il file (`:wq`)
Rendiamo il file eseguibile (`chmod +x newScript.sh`)
E runniamolo (`./newScript.sh`)

If - else

Proviamo ad aggiungere una condizione. Le condizione sono valutate in due modi:

- `[]` esegue la condizione contenuta
- `[[]]` supporta operatori come `&&`, `||`

Li usiamo per if - then - elif - else:

- if [condizione] then ...
- else ...

Proviamo ad usarli nel file di prima.

```
[mattia@pirandello prove]$ vim newScript.sh
```

Con G andiamo in fondo al file, e con O scriviamo dalla riga seguente

If - else

```
#!/bin/bash
TIMESTART=$(date)
echo "Script started by ${USER} at ${TIMESTART}"
MESSAGE=$({ls /pollegsasso} 2>&1 )
if [[ -n "${MESSAGE}" ]]; then
    echo "Warning: Errors were detected during script execution."
else
    echo "Script completed successfully with no errors."
fi
```

More work to do

Sia bash che vim sono programmi immensi che non si finisce mai di conoscerli a fondo. Di seguito sono riportati alcuni comandi che potete approfondire:

- `awk`, `sed`, `tr`
- `diff`
- `ip` e `ping`
- `rsync`
- `killall`
- `dd`
- `du`, `df`

ATTENZIONE: Mai usare un comando prima di sapere cosa fa. Alcuni di questi possono anche essere pericolosi se usati senza un minimo di criterio.

>ADM
staff

LABORATORI FRA PARI

AA 2025-2026

TERMINALE 1

04/11 17:00-19:00
AULA PINCHERLE
Dip. Matematica

TERMINALE 2

06/11 16:00-18:00
AULA E2
via mura Anteo Zamboni 2B

GIT 1 — COMANDI BASE

13/11 16:00-18:00
AULA E2
via mura Anteo Zamboni 2B

GIT 2 — CONDIVISIONE REMOTA

20/11 16:00-18:00
AULA E2
via mura Anteo Zamboni 2B

GITHUB 1

ISSUE
PULL REQUEST
CONFLITTI

27/11 16:00-18:00
AULA E2
via mura Anteo Zamboni 2B

GITHUB 2

CI/CD
ACTIONS
GIT LFS

13/11 16:00-18:00
AULA E2
via mura Anteo Zamboni 2B

PORTA IL TUO PC

PORTA LA TUA CURIOSITA'